LEA LABS · DIGITAL SKILLS

Campus Connect: Student Lab

A practical course pack for building, testing, and explaining modern web experiences.

Designed for teaching and learning

Code samples · guided practice · testing habits · inclusive interfaces

LEARN · BUILD · PUBLISH

LEA LABS

Student Lab Assignment: Campus Connect

Build, test, and publish a responsive events board

Module: Web Development Fundamentals
Project: Campus Connect
Recommended duration: 2–3 weeks
Submission: Project folder or repository, live link if published, README, and a short demonstration
Total: 100 marks

> **The brief.** Build a small, useful web experience that helps learners discover campus events and reserve a place. Your implementation must use semantic HTML, responsive CSS, and vanilla JavaScript. The reservation flow is a browser-side demonstration; it must not claim to store real bookings unless you have implemented and documented a secure server-side system.

1. Scenario

LEA Labs wants a simple public page where learners can discover workshops, career sessions, and community events. The first version should make the next action obvious. A visitor should understand what Campus Connect is, browse upcoming events, narrow the list by category, and submit a reservation request with useful feedback.

You are the developer responsible for the first working version. The visual design should be clear and intentional, but the assessment focuses on the quality of the structure, behavior, accessibility, testing, and explanation rather than decorative complexity.

2. Learning goals

This lab assesses whether you can apply the core front-end skills from the module. You should be able to structure a document with semantic HTML, create a flexible layout with CSS Grid and Flexbox, model repeated content as JavaScript data, respond to user events, validate a form, and test the result with a keyboard and multiple viewport sizes.

3. Required deliverables

Deliverable	Minimum requirement
`index.html`	Semantic page structure with a header, navigation, main content, event section, reservation section, and footer
`styles.css`	LEA-inspired or clearly documented visual system, responsive layout, visible focus states, and no horizontal overflow at required test widths
`script.js`	At least six event objects, dynamic card rendering, category filtering, and reservation form feedback
README	Project purpose, setup instructions, feature list, known limitations, test checklist, and a short explanation of two technical decisions
Evidence	Two screenshots or a short screen recording showing the page at different widths and a working interaction
Demonstration	A 3–5 minute walkthrough or live presentation in which you explain the data model, one event flow, and one debugging decision

4. Functional specification

4.1 Event data and cards

Create at least six event objects. Each object must include a unique identifier, title, category, date, and description. Render the cards from the data rather than duplicating six separate card structures in the HTML. The rendered card must expose the event title as a heading and make the category and date readable.

At minimum, use three categories such as `workshop`, `career`, and `community`. The category names should be consistent between the data, filter controls, and visible card content.

4.2 Category filters

Provide an “All” control and at least three category controls. Clicking a control must update the visible event cards without reloading the page. The selected control must have a visible state. If you use a toggle-style button, update its `aria-pressed` value consistently. If a category has no results, show a clear empty-state message rather than a blank region.

4.3 Reservation form

Provide a form with a labelled name field, a labelled email field, and a submit button. Use native HTML constraints such as `required` and `type="email"`. When the browser considers the form valid, handle the submit event in JavaScript, prevent the demo page reload, and show a confirmation message in an `aria-live` or `role="status"` region. Make it clear to the user that the current version records interest locally or is a classroom demonstration. Do not describe the flow as a confirmed real booking.

4.4 Responsive experience

The page must remain usable at approximately 320px, 768px, and 1200px viewport widths. The event grid should adapt without requiring horizontal scrolling. Navigation and form controls may stack at narrow widths. Images, if used, must remain within their containers. Record the widths you tested in the README.

4.5 Accessibility and quality

The page must include a sensible heading hierarchy, landmarks, labels, keyboard-reachable controls, visible focus, readable contrast, and clear feedback. Do not remove the focus outline unless you replace it with a visible equivalent. A clean browser console is part of the quality bar. If a known warning remains, document it and explain why.

5. Recommended implementation plan

Phase 1 — Sketch and structure. Draw the page on paper or in a simple wireframe. Identify the primary user action. Write the semantic HTML and make sure the page reads sensibly before CSS.

Phase 2 — Style and resize. Add design tokens, a container, typography, event cards, filter controls, form layout, and focus states. Check a narrow viewport before polishing desktop spacing.

Phase 3 — Model and render. Create the event array and a `renderEvents(items)` function. Render an empty state when the filtered list contains no events.

Phase 4 — Filter and submit. Add event listeners to filter controls. Update the rendered list and selected state. Add form submission feedback using `FormData` and a status region.

Phase 5 — Test and document. Use the keyboard, resize the viewport, submit invalid and valid input, inspect the console, and record at least three test results. Update the README and prepare the demonstration.

6. Suggested test record

Test	Expected result	Actual result	Status
Load at 320px	No horizontal scrolling; content remains readable		
Press Tab through controls	Every control is reachable and focus is visible		
Click “All”	All events appear		
Click a category	Only matching events appear		
Choose a category with no matches	Helpful empty-state message appears		
Submit empty form	Browser identifies required fields		
Submit invalid email	Browser identifies invalid format		
Submit valid form	Status message appears without a page reload		
Inspect console	No unresolved errors		

7. Extension challenges

Choose one extension only after all required features work. You may add a search input that combines with the category filter; display the number of currently visible events; add a favourites interaction; sort events by date; or persist the selected filter with `localStorage`. If you use `localStorage`, explain what happens when the stored value is missing or invalid.

A more advanced extension can load event data from a JSON file or endpoint. If you choose this route, add loading, empty, and error states. Explain why data received from outside the page should not be inserted into the DOM as unsanitized HTML.

8. Submission checklist

Before submitting, confirm that the project opens from a clean folder, the three source files are linked correctly, the README explains how to run the project, and no secrets or personal data are included. Test the final version at the required widths and with the keyboard. Verify that the live link, if provided, works in a private browser window. Make at least four meaningful commits if using Git, and ensure your commit messages describe the change.

9. Grading rubric — 100 marks

Criterion	Marks	Full-credit evidence
Semantic HTML and structure	18	Landmarks, heading hierarchy, meaningful sections, correctly associated labels, appropriate links and buttons
Responsive CSS and visual system	18	Flexible event grid, sensible narrow-screen behavior, readable type, coherent spacing and colour tokens, no required-width overflow
Data model and dynamic rendering	15	Six or more event objects with consistent fields and cards generated from data
Filtering behavior	15	All plus three categories work without reload, no-result state is handled, active state is clear and consistent
Form validation and feedback	12	Native constraints work, valid submit is handled in JavaScript, status message is visible and accessible, demo limitation is stated
Accessibility and keyboard UX	10	Controls are keyboard reachable, focus is visible, labels and headings are clear, contrast and status feedback are considered
Debugging and test evidence	6	At least three reproducible tests or bug records with expected and actual results
Documentation and demonstration	6	README is runnable and clear; learner explains two decisions and one debugging episode
Total	**100**	

Performance bands

Distinction (80–100). The project is complete, responsive, accessible, and easy to understand. The learner can explain how data flows through the page, why the layout behaves at different widths, and how a defect was diagnosed. Extensions are purposeful rather than decorative.

Merit (65–79). All major features work and the project is documented. Minor inconsistencies may remain in styling, accessibility, or edge-case handling, but the learner demonstrates a sound understanding of the implementation.

Pass (50–64). The core page loads and demonstrates substantial HTML, CSS, and JavaScript work, but one or more required interactions, responsive behaviors, or documentation elements are incomplete.

Not yet achieved (0–49). The project is missing major deliverables, does not run reliably, or cannot be explained and tested by the learner. Resubmission should focus on a smaller working version before adding visual complexity.

10. Academic integrity and collaboration

You may discuss ideas and use official documentation, but your submitted code must be work you can explain. If you use a tutorial, template, library, or generative assistant, record that in the README and identify which parts you changed. Do not copy personal data, credentials, or code that you cannot test. During demonstration, be prepared to modify one label, category, or event live.

11. Reflection prompts

After the demonstration, write a short response to the following questions. Which semantic element improved your page most? What was the first responsive failure you observed? How does the filter know which category to apply? Why is client-side validation not enough for a real booking system? What would you improve if the events came from a server instead of a local array?

References

[1] MDN Web Docs — Responsive web design, used for flexible grids, media queries, and mobile-first responsive practice.
[2] MDN Web Docs — Client-side form validation, used for native constraints, feedback, and the limits of client-side validation.
[3] MDN Web Docs — Introduction to events, used for event listeners, event objects, and default behavior.